

I. Pen and paper

1.

1

- Treino : x_1 a x_6
- Teste : x_7 e x_8
- input : y_1 a y_5
- target : y_6 (classes N ou P)
- conjuntos independentes : $\{y_1, y_2\}, \{y_3, y_4\}, \{y_5\}$
 $(y_1, y_2) \perp (y_3, y_4) \perp y_5$

• (y_1, y_2) segue distribuição Normal Bivariada

a) Priors para dados de Treino (x_1 a x_6)

classe N : $x_1, x_2, x_3 \rightarrow 3$ observações $\Rightarrow P(N) = \frac{3}{6} = 0,5$

classe P : $x_4, x_5, x_6 \rightarrow 3$ observações $\Rightarrow P(P) = \frac{3}{6} = 0,5$

Parâmetros para $\{y_1, y_2\}$ - Distribuição Normal Bivariada

classe N: valor das médias

$$\mu_N = \begin{bmatrix} \frac{0,52 + 0,53 + 0,42}{3} \\ \frac{0,80 + 0,92 + 0,48}{3} \end{bmatrix} = \begin{bmatrix} 0,49 \\ 0,73 \end{bmatrix}$$

Matriz de covariância

$$\text{Var}(y_1) = \frac{(0,52 - 0,49)^2 + (0,53 - 0,49)^2 + (0,42 - 0,49)^2}{3 - 1} = 0,0037$$

$$\text{Var}(y_2) = \frac{(0,80 - 0,73)^2 + (0,92 - 0,73)^2 + (0,48 - 0,73)^2}{3 - 1} = 0,05175$$

$$\text{cov}(g_1, g_2) = \frac{(0,52-0,49)(0,8-0,71) + (0,53-0,49)(0,92-0,71) + (0,42-0,49)(0,62-0,71)}{3-1}$$

$$= 0,0136$$

$$\Sigma_N = \begin{bmatrix} 0,0037 & 0,0136 \\ 0,0136 & 0,05175 \end{bmatrix}$$

classe P:
Vetor das médias

$$\mu_P = \begin{bmatrix} \frac{0,49 + 0,62 + 0,44}{3} \\ \frac{0,58 + 0,31 + 0,38}{3} \end{bmatrix} = \begin{bmatrix} 0,517 \\ 0,423 \end{bmatrix}$$

Matriz de covariâncias

$$\text{var}(g_1) = \frac{(0,49-0,517)^2 + (0,62-0,517)^2 + (0,44-0,517)^2}{3-1} = 0,00863$$

$$\text{var}(g_2) = \frac{(0,58-0,423)^2 + (0,31-0,423)^2 + (0,38-0,423)^2}{3-1} = 0,01963$$

$$\text{cov}(g_1, g_2) = \frac{(0,49-0,517)(0,58-0,423) + (0,62-0,517)(0,31-0,423) + (0,44-0,517)(0,38-0,423)}{3-1}$$

$$= -0,00628$$

$$\Sigma_P = \begin{bmatrix} 0,00863 & -0,00628 \\ -0,00628 & 0,01963 \end{bmatrix}$$

Parâmetro para $\{y_3, y_4\}$ - variáveis discretas

classe N:

	y_3	y_4
x_1	0	1
x_2	0	0
x_3	0	1

$$P(y_3 = 0 | N) = \frac{3}{3} = 1$$

$$P(y_3 = 1 | N) = \frac{0}{3} = 0$$

$$P(y_4 = 0 | N) = \frac{1}{3}$$

$$P(y_4 = 1 | N) = \frac{2}{3}$$

classe P

	y_3	y_4
x_4	1	0
x_5	0	0
x_6	0	0

$$P(y_3 = 0 | P) = \frac{2}{3}$$

$$P(y_3 = 1 | P) = \frac{1}{3}$$

$$P(y_4 = 0 | P) = \frac{3}{3} = 1$$

$$P(y_4 = 1 | P) = \frac{0}{3} = 0$$

Parâmetro p. de y_s - variável binária

classe N:

	y_s
x_1	1
x_2	0
x_3	1

$$P(y_s = 0 | N) = \frac{1}{3}$$

$$P(y_s = 1 | N) = \frac{2}{3}$$

classe P:

	y_s
x_4	1
x_5	1
x_6	1

$$P(y_s = 0 | P) = \frac{0}{3} = 0$$

$$P(y_s = 1 | P) = \frac{3}{3} = 1$$

b) Para $x_7 = \{0,45; 0,80; 0; 0; 1\}$

classe N

$$P(N | x_7) = P(N) \times N([0,45; 0,80]; \mu_N, \Sigma_N) \times P(y_1 = 0 | N) \times \\ \times P(y_4 = 0 | N) \times P(y_5 = 1 | N)$$

$$N(x; \mu, \Sigma) = \frac{1}{2\pi \sqrt{|\Sigma|}} \times e^{\left(-\frac{1}{2} (x - \mu)^T \cdot \Sigma^{-1} (x - \mu)\right)}$$

$$\det(\Sigma_N) = 0,0037 \times 0,05175 - 0,0136^2 = 0,00000652$$

$$\sqrt{|\Sigma_N|} = 0,00255$$

$$\Sigma_N^{-1} = \frac{1}{6,52 \times 10^{-6}} \begin{bmatrix} 0,05175 & -0,0136 \\ -0,0136 & 0,0037 \end{bmatrix} = \begin{bmatrix} 7937,1 & -2085,9 \\ -2085,9 & 561,5 \end{bmatrix}$$

$$x - \mu_N = \begin{bmatrix} 0,45 - 0,49 \\ 0,80 - 0,73 \end{bmatrix} = \begin{bmatrix} -0,04 \\ 0,067 \end{bmatrix}$$

$$(x - \mu_N)^T \Sigma_N^{-1} (x - \mu_N) = \begin{bmatrix} -0,04 & 0,067 \end{bmatrix} \begin{bmatrix} 7937,1 & -2085,9 \\ -2085,9 & 561,5 \end{bmatrix} \begin{bmatrix} -0,04 \\ 0,067 \end{bmatrix}$$

$$= 26,43$$

$$N([0,45; 0,8] | \mu_N, \Sigma_N) = \frac{1}{2\pi \times 0,00255} \cdot e^{\left(-\frac{1}{2} \times 26,43\right)}$$

$$= 1,14 \times 10^{-5}$$

$$P(N|a_1) = 0,5 \times 1,14 \times 10^{-5} \times 1 \times \frac{1}{3} \times \frac{2}{3} = 1,27 \times 10^{-6}$$

Classe P

$$\det(\tilde{\Sigma}_P) = 0,00863 \times 0,01963 - (-0,00628)^2 = \\ = 1,3 \times 10^{-4}$$

$$\sqrt{|\tilde{\Sigma}_P|} = 0,0114$$

$$\tilde{\Sigma}_P^{-1} = \frac{1}{1,3 \times 10^{-4}} \begin{bmatrix} 0,01963 & 0,00628 \\ 0,00628 & 0,00863 \end{bmatrix} =$$

$$= \begin{bmatrix} 151 & 48,3 \\ 48,3 & 66,4 \end{bmatrix}$$

$$\eta - \mu_P = \begin{bmatrix} 0,45 - 0,517 \\ 0,80 - 0,423 \end{bmatrix} = \begin{bmatrix} -0,067 \\ 0,377 \end{bmatrix}$$

$$(\eta - \mu_P)^T \tilde{\Sigma}_P^{-1} (\eta - \mu_P) = \begin{bmatrix} -0,067 & 0,377 \end{bmatrix} \begin{bmatrix} 151 & 48,3 \\ 48,3 & 66,4 \end{bmatrix} \begin{bmatrix} -0,067 \\ 0,377 \end{bmatrix}$$

$$= 7,675$$

$$N\left(\begin{bmatrix} 0,45 \\ 0,8 \end{bmatrix}; \mu_P; \tilde{\Sigma}_P\right) = \frac{1}{2\pi \times 0,0114} e^{\left(-\frac{1}{2} \times 7,675\right)} = 0,301$$

$$P(P|\mathcal{I}_7) = P(P) \times N(\eta; \mu_P; \tilde{\Sigma}_P) \times P(y_3=0|P) \times P(y_4=0|P) \times P(y_5=1|P)$$

$$P(R|\mathcal{I}_7) = 0,5 \times 0,301 \times \frac{2}{3} \times 1 \times 1 = 0,1003$$

coms $P(P|x_1) \gg P(N|x_1)$ x_1 é classificado coms P

$$x_8 = [0,5; 0,7; 0; 1; 1]$$

donc N

$$\mu - \mu_N = \begin{bmatrix} 0,5 - 0,49 \\ 0,7 - 0,733 \end{bmatrix} = \begin{bmatrix} 0,01 \\ -0,033 \end{bmatrix}$$

$$(\mu - \mu_N)^T \Sigma_N^{-1} (\mu - \mu_N) = [0,01 \quad -0,033] \begin{bmatrix} 7337,1 & -2085,9 \\ -2085,9 & 567,5 \end{bmatrix} \begin{bmatrix} 0,01 \\ -0,033 \end{bmatrix}$$

$$= 2,788$$

$$N([0,5; 0,7]; \mu_N; \Sigma_N) = \frac{1}{2\pi \times 0,00255} e^{(-\frac{1}{2} \times 2,788)}$$

$$= 15,48$$

$$P(N|x_8) = P(N) \times N(x_8, \mu_N, \Sigma_N) \times P(y_1=0|N) \times P(y_4=1|N) \times P(y_5=1|N)$$

$$= 0,15 \times 15,48 \times 1 \times \frac{2}{3} \times \frac{2}{3} = 3,44$$

classe P

$$P(P|x_1) = P(P) \times N(x_1, \mu_P, \sigma_P^2) \times P(y_1=0|P) = \frac{11}{0} \times P(y_1=1|P) \times P(y_2=1|P)$$

$$\log P(P|x_1) = 0$$

como $P(N|x_1) \gg P(P|x_1)$ x_1 é classificado como N

obs	y_i	$\hat{z}$	Resultado
x_1	P	P	✓
x_2	N	N	✓

Accuracy no teste é 100%.

2.

(2)

	y_3	y_4	y_5	y_6
x_1	0	1	1	N
x_2	0	0	0	N
x_3	0	1	1	N
x_4	1	0	1	P
x_5	0	0	1	P
x_6	0	0	1	P
x_7	0	0	1	P
x_8	0	1	1	N

d_1 :

$$d(x_1, x_2) = 2$$

$$d(x_1, x_3) = (0)$$

$$d(x_1, x_4) = 2$$

$$d(x_1, x_5) = 1$$

$$d(x_1, x_6) = 1$$

$$d(x_1, x_7) = 1$$

$$d(x_1, x_8) = (0)$$

vizando x_1, x_8
 $\downarrow \quad \downarrow$
 N N

Predição para $x_1 = N$ ✓

x_2 :

$$d(x_2, x_1) = 2$$

$$d(x_2, x_3) = 2$$

$$d(x_2, x_4) = 2$$

$$d(x_2, x_5) = \textcircled{1}$$

$$d(x_2, x_6) = \textcircled{1}$$

$$d(x_2, x_7) = \textcircled{1}$$

$$d(x_2, x_8) = 2$$

3 vizinhos mais próximos: $x_5(P)$; $x_6(P)$; $x_7(P)$

Predecessor para x_2 : $P \times$

x_3 :

$$d(x_3, x_1) = \textcircled{0}$$

$$d(x_3, x_2) = 2$$

$$d(x_3, x_4) = 2$$

$$d(x_3, x_5) = 1$$

$$d(x_3, x_6) = 1$$

$$d(x_3, x_7) = 1$$

$$d(x_3, x_8) = \textcircled{0}$$

2 vizinhos mais próximos: $x_1(N)$; $x_8(N)$

Predecessor para x_3 : $N \checkmark$

x_4 :

$$d(x_4, x_1) = 2$$

$$d(x_4, x_2) = 2$$

$$d(x_4, x_3) = 2$$

$$d(x_4, x_5) = \textcircled{1}$$

$$d(x_4, x_6) = \textcircled{1}$$

$$d(x_4, x_7) = \textcircled{1}$$

$$d(x_4, x_8) = 2$$

3 vizinhos mais próximos: $x_5(P)$; $x_6(P)$; $x_7(P)$

Predecessor para x_4 : $P \checkmark$

x_5 :

$$d(x_5, x_1) = 1$$

$$d(x_5, x_2) = 1$$

$$d(x_5, x_3) = 1$$

$$d(x_5, x_4) = 1$$

$$d(x_5, x_6) = 0$$

$$d(x_5, x_7) = 0$$

$$d(x_5, x_8) = 1$$

2 vizinhos mais próximos: $x_6(P)$; $x_7(P)$

Predefinição para x_5 : P ✓

x_6 :

$$d(x_6, x_1) = 1$$

$$d(x_6, x_2) = 1$$

$$d(x_6, x_3) = 1$$

$$d(x_6, x_4) = 1$$

$$d(x_6, x_5) = 0$$

$$d(x_6, x_7) = 0$$

$$d(x_6, x_8) = 1$$

2 vizinhos mais próximos: $x_5(P)$; $x_7(P)$

Predefinição para x_6 : P ✓

x_7 :

$$d(x_7, x_1) = 1$$

$$d(x_7, x_2) = 1$$

$$d(x_7, x_3) = 1$$

$$d(x_7, x_4) = 1$$

$$d(x_7, x_5) = 0$$

$$d(x_7, x_6) = 0$$

$$d(x_7, x_8) = 1$$

2 vizinhos mais próximos: $x_5(P)$; $x_6(P)$

Predefinição para x_7 : P ✓

x_8

$$d(x_8, x_1) = 0$$

$$d(x_8, x_4) = 2$$

$$d(x_8, x_2) = 2$$

$$d(x_8, x_5) = 1$$

$$d(x_8, x_3) = 2$$

$$d(x_8, x_6) = 1$$

$$d(x_8, x_7) = 1$$

2 vizinhos mais próximos: $x_1(N)$; $x_7(N)$

Predição para x_8 : N ✓

Resultado final

7 predições corretas em 8

$$\text{Accuracy} = \frac{7}{8} = 87,5\%$$

3. a) $P(\theta=0) = p \quad p \in [1/2, 1]$

$$P(\theta=1) = 1-p$$

$$P(X=x | \theta=0) = P(X=x | \theta=1) \quad \forall x$$

$$P(X=x) > 0 \quad \forall x$$

Usando o Teorema de Bayes:

$$P(\theta=0 | X=x) = \frac{P(X=x | \theta=0) \cdot P(\theta=0)}{P(X=x)}$$

$$P(\theta=1 | X=x) = \frac{P(X=x | \theta=1) \cdot P(\theta=1)}{P(X=x)}$$

Comme $P(X=x | \theta=0) = P(X=x | \theta=1)$ alors :

$$P(\theta=0 | X=x) = \frac{P(X=x | \theta=0) \times p}{P(X=x)}$$

$$P(\theta=1 | X=x) = \frac{P(X=x | \theta=1) \times (1-p)}{P(X=x)}$$

$$\frac{P(\theta=0 | X=x)}{P(\theta=1 | X=x)} = \frac{p}{(1-p)}$$

Comme $p > 1/2$ alors $p > 1-p$ donc

$$P(\theta=0 | X=x) > P(\theta=1 | X=x)$$

Par conséquent, le classificateur Bayésien va toujours prédire

$\theta_{\text{Bayes}} = 0$ (indépendamment de la valeur de x)

Erreur de Bayes :

$$E_{\text{Bayes}} = P(\theta \neq \theta_{\text{Bayes}}) = P(\theta \neq 0) = P(\theta=1) = 1-p //$$

b) Quand $n \rightarrow \infty$, l'erreur de 1-NN est doublée par :

$$E_{1\text{NN}} = 2 \times P(\theta=0 | X=x) \times P(\theta=1 | X=x)$$

Comme $P(X=x | \theta=0) = P(X=x | \theta=1)$ nous cherchons cette égalité de $L(x)$

Per Teorema di Bayes

$$P(\theta = 0 | X=x) = \frac{L(x) \times P}{L(x) \times P + L(x) \times (1-P)} = \frac{P}{P + (1-P)} = P$$

$$P(\theta = 1 | X=x) = \frac{L(x) \times (1-P)}{L(x) \times P + L(x) \times (1-P)} = \frac{1-P}{P + (1-P)} = 1-P$$

Portante

$$Z \times P(\theta = 0 | X=x) \times P(\theta = 1 | X=x) = Z \times P \times (1-P) = ZP(1-P) //$$

$$c) E_{\text{Bayes}} = 1-P \quad (\text{almeno } a)$$

$$E_{\text{INN}} = ZP(1-P) \quad (\text{almeno } b)$$

Sostituendo E_{Bayes} per $1-P$:

$$Z \times E_{\text{Bayes}} \times (1 - E_{\text{Bayes}}) = Z \times (1-P) \times (1 - (1-P)) =$$

$$= Z \times (1-P) \times P = ZP(1-P) = E_{\text{INN}} //$$

II. Programming

1. a)

```
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
import pandas as pd
import numpy as np

# Carregar e limpar dataset
df = pd.read_csv('Breast_cancer_dataset.csv')
df = df.loc[:, ~df.columns.str.contains('^Unnamed')]

# Separar features e target
X = df.drop(['id', 'diagnosis'], axis=1)
y = df['diagnosis']

# Inicializar modelos
knn = KNeighborsClassifier(n_neighbors=5)
gnb = GaussianNB()

# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
knn_scores = cross_val_score(knn, X, y, cv=cv, scoring='accuracy')
gnb_scores = cross_val_score(gnb, X, y, cv=cv, scoring='accuracy')

# Resultados
print("kNN (k=5) scores:", knn_scores)
print(f"kNN Mean: {knn_scores.mean():.4f}, Std: {knn_scores.std():.4f}")

print("\nNaive Bayes scores:", gnb_scores)
print(f"Naive Bayes Mean: {gnb_scores.mean():.4f}, Std: {gnb_scores.std():.4f}")
```

✓ 0.2s Python

kNN (k=5) scores: [0.92105263 0.93859649 0.92105263 0.9122807 0.96460177]
kNN Mean: 0.9315, Std: 0.0186

Naive Bayes scores: [0.92105263 0.96491228 0.9122807 0.93859649 0.95575221]
Naive Bayes Mean: 0.9385, Std: 0.0199

Uma vez que o desvio padrão do k-Nearest Neighbors é ligeiramente menor, podemos afirmar que este classificador é mais estável.

Isto deve-se, provavelmente, ao facto de existirem features correlacionadas no dataset (por exemplo, `radius_mean` e `area_mean`), que são melhor tratadas pelo kNN, que as interpreta como pontos no espaço a fim de procurar casos similares numa vizinhança, do que pelo Gaussian Naive Bayes, que assume sempre independência entre features e que acaba, assim, por desenhar novas suposições probabilísticas a partir de informação que, na realidade, está correlacionada com uma outra qualquer feature.

b)

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline

# Pipeline com scaler para kNN
knn_scaled = Pipeline([
    ('scaler', MinMaxScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

# Cross-validation com kNN escalado
knn_scaled_scores = cross_val_score(knn_scaled, X, y, cv=cv, scoring='accuracy')

# Resultados
print("\nkNN Scaled (k=5) scores:", knn_scaled_scores)
print(f"kNN Scaled Mean: {knn_scaled_scores.mean():.4f}, Std: {knn_scaled_scores.std():.4f}")

print("\nComparação:")
print(f"kNN sem scaling: {knn_scores.mean():.4f}")
print(f"kNN com scaling: {knn_scaled_scores.mean():.4f}")
print(f"Diferença: {(knn_scaled_scores.mean() - knn_scores.mean()):.4f}")
```

Python

kNN Scaled (k=5) scores: [0.93859649 0.99122807 0.97368421 0.97368421 0.98230088]
kNN Scaled Mean: 0.9719, Std: 0.0179

Comparação:
kNN sem scaling: 0.9315
kNN com scaling: 0.9719
Diferença: 0.0404

Dado que o classificador kNN depende da distância entre os datapoints para realizar a classificação, variáveis com maiores amplitudes podem influenciar desproporcionalmente os cálculos de distância, gerando classificações enviesadas.

Através do escalonamento Min-Max, que transforma todas as variáveis para uma escala comum, é possível eliminar este enviesamento e, assim, consegue-se obter um modelo de melhor desempenho, mais fiável e mais estável, conforme podemos ver pelos resultados obtidos: não só a accuracy média é maior, como o desvio padrão entre as accuracies é menor.

c)

```
from scipy.stats import ttest_rel

# Naive Bayes também com scaling (para comparação justa)
gnb_scaled = Pipeline([
    ('scaler', MinMaxScaler()),
    ('gnb', GaussianNB())
])

gnb_scaled_scores = cross_val_score(gnb_scaled, X, y, cv=cv, scoring='accuracy')

# Paired t-test
statistic, p_value = ttest_rel(knn_scaled_scores, gnb_scaled_scores)

print("\nPaired t-test:")
print(f"kNN (scaled) scores: {knn_scaled_scores}")
print(f"GNB (scaled) scores: {gnb_scaled_scores}")
print(f"t-statistic: {statistic:.4f}")
print(f"P-value: {p_value:.4f}")
print(f"Alpha: 0.05")

if p_value < 0.05:
    if knn_scaled_scores.mean() > gnb_scaled_scores.mean():
        print(f"\nConclusão: kNN é estatisticamente superior (p={p_value:.4f} < 0.05)")
    else:
        print(f"\nConclusão: Naive Bayes é estatisticamente superior (p={p_value:.4f} < 0.05)")
else:
    print(f"\nConclusão: Não há diferença estatisticamente significativa (p={p_value:.4f} >= 0.05)")
```

Python

```
Paired t-test:
kNN (scaled) scores: [0.93859649 0.99122807 0.97368421 0.97368421 0.98230088]
GNB (scaled) scores: [0.9122807  0.93859649 0.92105263 0.94736842 0.92920354]
t-statistic: 6.5075
P-value: 0.0029
Alpha: 0.05
```

Conclusão: kNN é estatisticamente superior (p=0.0029 < 0.05)

A fim de testar a hipótese de que o classificador kNN é estatisticamente superior ao Naive Bayes, foi realizado um paired t-test, partindo das seguintes hipóteses:

- Hipótese nula (H_0): A precisão do kNN não é diferente da do Naive Bayes;
- Hipótese alternativa (H_1): A precisão do kNN é diferente da do Naive Bayes.

Analisando os resultados acima, é possível verificar que o p-value obtido é inferior a 0.05, indicando-nos que a probabilidade das diferenças de performance verificadas entre os dois classificadores terem ocorrido como fruto do acaso é inferior a 5%, demonstrando, assim, que os resultados são estatisticamente significativos e que, portanto, é possível confirmar que o modelo kNN é superior ao Naive Bayes.

2. a)

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import pandas as pd
import matplotlib.pyplot as plt

# Carregar e limpar dataset
df = pd.read_csv('Breast_cancer_dataset.csv')
df = df.loc[:, ~df.columns.str.contains('^Unnamed')]

# Features e target
X = df.drop(['id', 'diagnosis'], axis=1)
y = df['diagnosis']

# 70-30 train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

# Valores de k
k_values = [1, 5, 10, 15, 20, 25]

# Listas para armazenar accuracies
train_acc_uniform = []
test_acc_uniform = []
train_acc_distance = []
test_acc_distance = []

# Iterar sobre diferentes valores de k
for k in k_values:
    # kNN com uniform weights
    knn_uniform = KNeighborsClassifier(n_neighbors=k, weights='uniform')
    knn_uniform.fit(X_train, y_train)
    train_acc_uniform.append(knn_uniform.score(X_train, y_train))
    test_acc_uniform.append(knn_uniform.score(X_test, y_test))

    # kNN com distance weights
    knn_distance = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn_distance.fit(X_train, y_train)
    train_acc_distance.append(knn_distance.score(X_train, y_train))
    test_acc_distance.append(knn_distance.score(X_test, y_test))
```



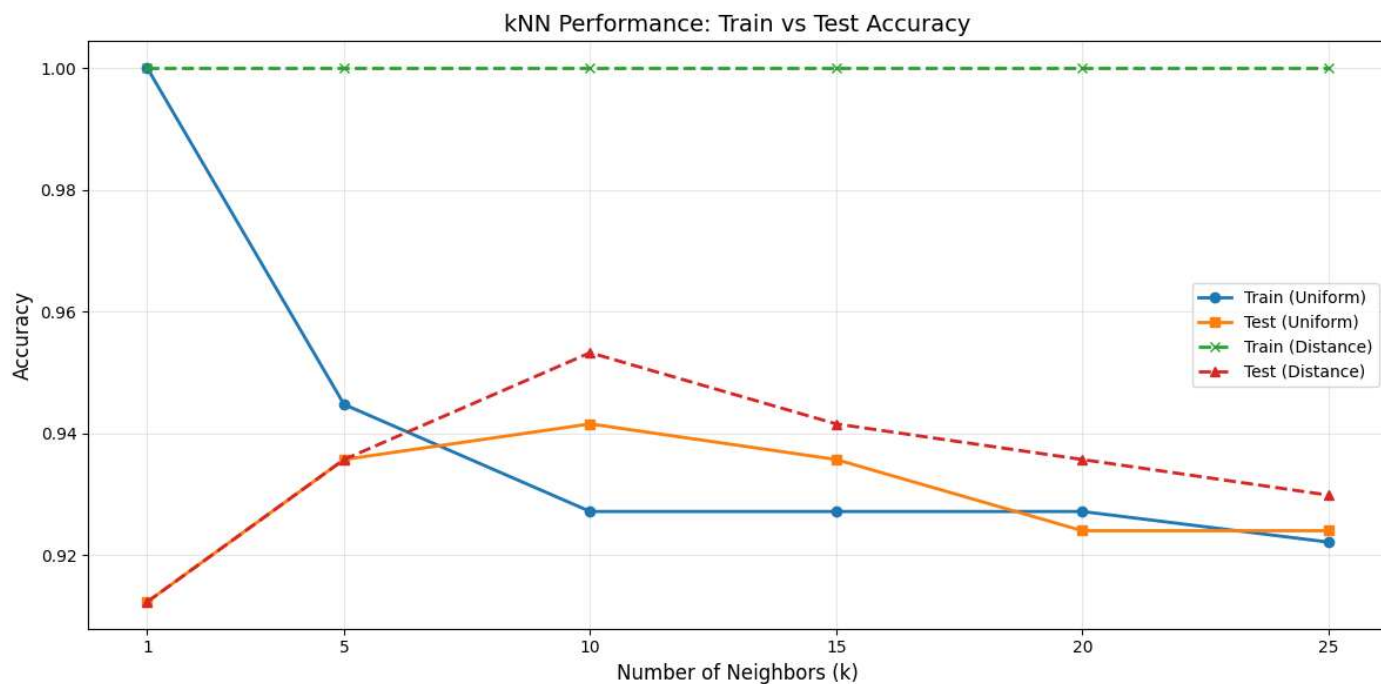
```
# Plot
plt.figure(figsize=(12, 6))

plt.plot(k_values, train_acc_uniform, 'o-', label='Train (Uniform)', linewidth=2)
plt.plot(k_values, test_acc_uniform, 's-', label='Test (Uniform)', linewidth=2)
plt.plot(k_values, train_acc_distance, 'x--', label='Train (Distance)', linewidth=2)
plt.plot(k_values, test_acc_distance, '^--', label='Test (Distance)', linewidth=2)

plt.xlabel('Number of Neighbors (k)', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('kNN Performance: Train vs Test Accuracy', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.xticks(k_values)
plt.tight_layout()
plt.show()

# Print resultados para análise
print("k\tTrain_Uni\tTest_Uni\tTrain_Dist\tTest_Dist")
for i, k in enumerate(k_values):
    print(f"{k}\t{train_acc_uniform[i]:.4f}\t{test_acc_uniform[i]:.4f}\t{train_acc_distance[i]:.4f}\t{test_acc_distance[i]:.4f}")
```

Python



b)

A partir da análise do plot acima, é possível verificar que, até certo ponto, o aumento do valor de k leva a uma melhor capacidade de generalização dos modelos, degenerando, depois.

O uso de valores de k muito pequenos, como 1 ou 2, aparenta causar overfitting, capturando-se particularidades dos dados de treino, como o seu ruído e a presença de outliers e obtendo-se, consequentemente, accuracies menores, quando comparado a valores de k entre a faixa dos 5 e os 20. Aqui verificam-se as melhores accuracies do modelo: valores superiores a 20 parecem levar a underfitting, conseguindo-se accuracies cada vez menores à medida que se aumenta o valor de k , uma vez que se perdem padrões importantes, já que o cálculo da média dos vários datapoints passa a acontecer numa bola demasiado grande do espaço.

3)

Tendo em conta o dataset em questão e os testes sobre ele realizados ao longo deste Homework, é possível referir alguns fatores que poderiam influenciar a escolha de um dos modelos estudados - kNN ou Naive Bayes - em detrimento do outro.

Por um lado, uma vez que as features do dataset estão correlacionadas, é possível inferir que o kNN possa ser uma melhor escolha, já que estas correlações violariam a presunção de independência entre features subjacente ao Naive Bayes, levando, assim, a uma redução na estabilidade deste modelo (como, de resto, se verificou em 1.a)).

Por outro, dado que o classificador Naive Bayes produz afirmações probabilísticas como output, tem-se, com ele, uma maior interpretabilidade do que com o kNN. Embora a proximidade de um caso com outros seja passível de ser discutida num contexto clínico, a transparência dada por um modelo probabilístico como o conseguido em Naive Bayes poderá ser de valor num contexto clínico.

Estes fatores, entre outros como o maior custo computacional de kNN face a Naive Bayes, poderão ser determinantes na escolha de um classificador ou de outro.